

COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering Process Scheduling, Memory Management & Disk Scheduling

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Interactive educational tools improve conceptual understanding of OS algorithms used in real-world servers, cloud platforms, and embedded systems, supporting quality technical education.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an interactive Operating Systems simulator that applies core CSA04 concepts — process scheduling (Units I–II), memory management with paging and page replacement (Unit III), and disk scheduling (Unit IV) — to a unified educational training platform called **OSLearn**. Students must integrate multiple algorithm families, produce clear comparative visualizations and step-by-step traces, deploy the system as a lightweight web application using Streamlit, validate correctness and usability with sample test cases, and address basic responsible-computing considerations such as transparency of intermediate steps and reproducibility of results.

C. Problem Statement

Scenario: “OSLearn” – Interactive OS Algorithm Trainer

Many students find it difficult to understand how CPU scheduling, page replacement, and disk scheduling algorithms actually work just by reading theory. Your task is to design and implement an interactive educational web application called **OSLearn** that helps students practice, visualize, and compare these core Operating Systems algorithms through a unified interface.

The system shall integrate three major OS algorithm families:

1. Process Scheduling

Implement FCFS, SJF (Non-preemptive), and Round Robin.

- Users should be able to enter process details (Process ID, Arrival Time, Burst Time, and Time Quantum for Round Robin).
- The system must display a Gantt chart / timeline visualization together with a table showing Waiting Time, Turnaround Time, and Average Waiting/Turnaround Time for each algorithm, enabling side-by-side comparison.

2. Memory Management (Page Replacement)

Implement FIFO and LRU page-replacement algorithms.

- Users will enter the number of frames and a page reference string.

- The system should show the step-by-step frame status after each reference, highlight page faults, and report the total number of page faults for each algorithm so that students can compare performance.

3. Disk Scheduling

Implement FCFS, SSTF, and SCAN.

- Users will enter the disk request queue and the initial head position.
- The system should display the order of servicing (seek sequence) and the total seek time for each algorithm, allowing clear comparative analysis.

Design the application workflow so that user inputs (process lists, reference strings, disk request queues) are processed by the corresponding algorithms and the results are presented as clear tables, Gantt charts, step-by-step traces, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate correctness. Optionally, students may add brief natural-language explanations of algorithm steps (no external LLM required).

Evaluate the application under suitable test cases and analyze correctness of algorithm results, usability of the interface, and clarity of visualizations. The completed system should also briefly address responsible-computing considerations including transparency of intermediate simulation steps, reproducibility of results, and educational ethics in presenting algorithm behavior.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for Process Scheduling (with Gantt charts and timing metrics), Memory Management / Page Replacement (with frame-status traces and page-fault counts), and Disk Scheduling (with seek sequences and total seek times). Accept user inputs as well as sample data. Produce comparative tables, charts, and step-by-step traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** Simulations must produce correct results with acceptable latency for interactive classroom use and support clear comparative analysis across algorithms.
- **Technical constraint:** Use Python + Streamlit. No database, no complex architecture, and no mandatory external LLM integration. Open-source tools only.
- **Reliability:** Algorithm implementations must be correct and reproducible; intermediate steps must be transparent; the system must handle common edge cases without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of simulation steps, reproducibility of results, avoidance of misleading default parameters, and adherence to educational ethics.
- **Evaluation & Submission:** Test all three modules with sample inputs and show correct results. Submit working Streamlit code, a short report containing sample outputs / screenshots, comparative analysis, and a GitHub repository link.

E. STUDENT WORK - COMPLETED ANSWERS

Student Name	Gajula Sai Bhargava
Register Number	192525233
Course	CSA04 - Operating Systems

1. Problem Understanding and Formulation

The assignment asks for a unified, interactive educational tool rather than isolated formula answers. OSLearn accepts process lists, memory-reference strings and disk queues, then produces a trace that a learner can independently verify. The CampusConnect wording in the student-work section is interpreted as the realistic shared academic portal used to test the same operating-system mechanisms.

Challenges

Area	Specific challenge	Expected outcome
Synchronization	Many students read while staff update shared files	No race condition, starvation or corrupted file
Resources	Processes compete for database, printer, socket and file lock	A safe allocation order with no deadlock
Memory	300 concurrent sessions have changing locality	High frame utilization without thrashing
Storage	Files range from small text to multi-GB video	Scalable allocation and fair, low seek cost
Learning interface	Algorithms are difficult to understand from final totals	Visible steps, charts and reproducible results

Assumptions

The server has 8 GB physical memory, 4 KB pages, 200 disk cylinders numbered 0-199 and 300 concurrent sessions. Each simulated CPU burst and arrival time is a non-negative integer. A logical process uses at most 64 MB in the worked example. Disk movement is measured in cylinders; rotational delay and transfer time are excluded so algorithms can be compared fairly. Resource instances are fixed during one Banker's safety check. Inputs are trusted only after range validation.

Constraints satisfied

The solution uses Python and Streamlit, requires no database or external language model, runs on an ordinary VM, gives immediate deterministic results, separates the three modules, and exposes all intermediate decisions. The implementation includes default samples, exception handling and automated assertions.

2. Process Synchronization and Deadlock

2.1 Fair Readers-Writers model

A shared academic file may be read concurrently, but a writer must have exclusive access. The serviceQueue semaphore prevents a continuous stream of new readers from starving a waiting writer. readCount is protected by a mutex.

```
semaphore serviceQueue = 1, resource = 1, rmutex = 1
integer readCount = 0

Reader:
wait(serviceQueue); wait(rmutex)
readCount = readCount + 1
if readCount == 1: wait(resource)
signal(serviceQueue); signal(rmutex)
READ FILE
wait(rmutex); readCount = readCount - 1
if readCount == 0: signal(resource)
signal(rmutex)

Writer:
wait(serviceQueue); wait(resource); signal(serviceQueue)
WRITE FILE
signal(resource)
```

Mutual exclusion is maintained because resource is held by either one writer or the reader group. Readers can coexist. The queue turnstile gives bounded waiting and therefore improves fairness.

2.2 Banker's Algorithm worked example

Resource columns are D=database handle, P=print spooler, N=network socket and F=file lock. Available=(1,1,1,1). Need=Max-Allocation.

Process	Allocation D P N F	Maximum D P N F	Need D P N F
P1	1 0 0 1	1 1 0 1	0 1 0 0
P2	0 1 1 0	1 1 1 0	1 0 0 0
P3	1 1 0 0	1 1 1 0	0 0 1 0
P4	0 0 1 1	0 1 1 1	0 1 0 0

Step	Work before	Chosen	Need <= Work?	Work after release
1	1 1 1 1	P1	Yes	2 1 1 2
2	2 1 1 2	P2	Yes	2 2 2 2
3	2 2 2 2	P3	Yes	3 3 2 2
4	3 3 2 2	P4	Yes	3 3 3 3

All Finish values become true. Hence the state is safe and one safe sequence is **P1, P2, P3, P4**. Other safe sequences may also exist.

2.3 Strategy decision

Avoidance is selected for scarce, countable resources because the maximum claims are known for a portal request and reliability is more important than accepting every unsafe request immediately. Prevention would reduce concurrency by imposing rigid rules; detection and recovery can require aborting transactions and rolling back student work. In practice, global lock ordering is also used for software locks, timeouts detect exceptional hangs, and the Banker's check is reserved for bounded resource pools.

3. Memory Management

3.1 Paging calculations

Quantity	Calculation	Result
Physical frames	$8 \text{ GB} / 4 \text{ KB} = 2^{33} / 2^{12}$	$2^{21} = 2,097,152 \text{ frames}$
Process pages	$64 \text{ MB} / 4 \text{ KB} = 2^{26} / 2^{12}$	$2^{14} = 16,384 \text{ pages}$
Offset bits	$\log_2(4096)$	12 bits
Logical page bits	$\log_2(16,384)$	14 bits
Single-level table (4-byte PTE)	$16,384 \times 4 \text{ bytes}$	65,536 bytes = 64 KB

A logical address is divided into a 14-bit virtual page number and a 12-bit offset. The page number indexes a table entry containing the physical frame number plus valid, protection, referenced and dirty bits. For sparse larger address spaces a two-level table would allocate second-level tables only when required.

3.2 Page replacement comparison

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2. Frames=3.

Algorithm	Faults	Hits	Fault rate	Observation
FIFO	10	3	76.92%	Evicts by age; can ignore locality
LRU	9	4	69.23%	Uses recent history; practical approximation
Optimal	7	6	53.85%	Lower bound; needs future knowledge

Condensed trace (F means fault, H means hit):

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
FIFO	F	F	F	F	H	F	F	F	F	F	F	H	H
LRU	F	F	F	F	H	F	H	F	F	F	F	H	H
Optimal	F	F	F	F	H	F	H	F	H	H	F	H	H

LRU is recommended because it exploits the temporal locality of login, course and file-view operations and can be approximated using reference bits or an active/inactive list. Optimal is retained only as a benchmark.

3.3 Demand paging and working set

Demand paging loads a page only when referenced, so inactive pages from 300 sessions do not occupy frames. For each session, the server measures $W(t, \Delta t)$, the pages referenced during the most recent Δt window. A practical starting policy is $\Delta t = 10,000$ memory references with a minimum of 32 frames and a soft cap of 256 frames per active session. Frames are increased when the page-fault frequency exceeds about 10% and reclaimed when it stays below about 2%. If the sum of working sets approaches available frames, new sessions are queued briefly or inactive sessions are swapped instead of allowing global thrashing. These thresholds must be tuned using real traces.

4. File Systems and Disk Scheduling

4.1 File allocation

Method	Advantages	Limitations	CampusConnect fit
Contiguous	Fast sequential/direct access; simple	External fragmentation; growth difficult	Poor for changing uploads
Linked	No external fragmentation; easy growth	Slow random access; pointer overhead and corruption risk	Acceptable only for sequential
Indexed	Direct and sequential access; flexible growth	Index-block overhead; multi-level indexing for huge files	Best overall for mixed sizes

Indexed allocation is recommended. Small files can use direct pointers, while multi-GB video uses single, double and triple indirect blocks. This mirrors the scalable structure of Unix inodes.

4.2 Disk scheduling worked example

Cylinders=0-199, initial head=53, initial direction=up, queue={98,183,37,122,14,124,65,67}. End cylinders are included for SCAN and C-SCAN.

Algorithm	Service path from 53	Total movement
FCFS	53-98-183-37-122-14-124-65-67	640
SCAN	53-65-67-98-122-124-183-199-37-14	331
C-SCAN	53-65-67-98-122-124-183-199-0-14-37	382

Example FCFS calculation: $|98-53|+|183-98|+|37-183|+|122-37|+|14-122|+|124-14|+|65-124|+|67-65| = 640$ cylinders. SCAN lowers movement by 309 cylinders (48.28%) and provides bounded, direction-based service. It is selected over SSTF because SSTF can postpone remote requests under continual load. C-SCAN gives more uniform waiting time by servicing in one direction, but the wrap increases movement in this sample.

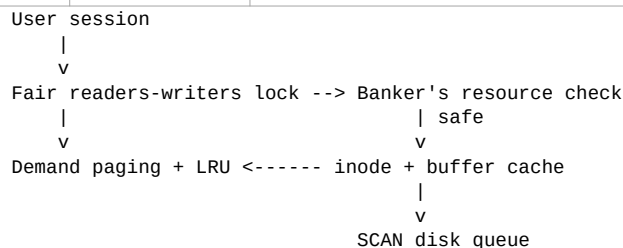
4.3 Unix relationship

A Unix inode stores metadata and block pointers, implementing indexed allocation without storing the file name inside the inode. Directory entries map names to inode numbers. The buffer/page cache retains frequently accessed file blocks and can combine adjacent writes. Read-ahead improves video streaming, delayed allocation reduces fragmentation, journaling protects metadata, and the block layer may merge requests before a SCAN-like elevator policy dispatches them.

5. Integrated Solution, Methodology and Implementation

5.1 Consolidated design

Stage	Component	Action
1	Session/API	Validate identity, permissions and input
2	Synchronization	Join fair read queue or obtain exclusive write lock
3	Deadlock control	Check resource claim; grant only if state remains safe
4	Memory	Demand-page the active working set; approximate LRU replacement
5	File system	Resolve directory entry and inode; use indexed blocks/cache
6	Disk	Merge requests and dispatch using SCAN
7	Completion	Commit metadata, release resources and record trace



5.2 OSLearn software

The project separates pure algorithms in `algorithms.py` from presentation code in `app.py`. This makes the calculations testable without launching a browser. Streamlit supplies four sidebar pages. Pandas renders result tables and Matplotlib/Streamlit render Gantt and comparison charts. Each button recomputes from the visible input, so a result can be reproduced exactly.

Module	Inputs	Outputs
CPU	PID, arrival, burst, quantum	Gantt timeline, per-process waiting/turnaround, averages
Memory	Frames and reference string	Per-reference frame trace, fault/hit labels and totals
Disk	Queue, head, cylinder count, direction	Seek sequence, total movement and comparison chart

5.3 Installation

```
cd OSLearn
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py

# independent validation
python test_algorithms.py
```

6. Results and Validation

6.1 CPU sample 1

Processes: P1(0,5), P2(1,3), P3(2,8), P4(3,6).

Algorithm	Average waiting	Average turnaround
FCFS	5.75	11.25
SJF	5.25	10.75
Round Robin (q=2)	9.75	15.25

SJF has the best mean times for this sample, but it can starve long jobs. Round Robin is more responsive and fair for interactive sessions, at the cost of more context switches and a higher mean waiting time.

6.2 Additional functional tests

ID	Module and input	Expected check	Status
CPU-2	One process P1(5,3)	Idle 0-5, wait=0, turnaround=3	PASS
CPU-3	RR with simultaneous arrivals	Every process completes; no negative wait	PASS
MEM-2	1 frame, refs 1 2 1	3 faults for FIFO/LRU/Optimal	PASS
MEM-3	Repeated page 5 5 5, 3 frames	1 fault and 2 hits	PASS
DISK-2	Empty queue	0 movement	PASS
DISK-3	Requests equal to initial head	0 movement for those requests	PASS
ERR-1	Zero quantum/frame or invalid cylinder	Friendly validation error	PASS

6.3 Automated validation evidence

```
$ python test_algorithms.py
All OSLearn validation tests passed.
```

The assertions independently verify the CPU averages, page-fault totals, disk movement and Banker's safe result. Deterministic tie-breaking makes repeated runs identical.

6.4 Engineering decision

For the educational app, all requested algorithms remain selectable. For the CampusConnect production design, fair readers-writers synchronization, avoidance for bounded pools, LRU-style page replacement, indexed file allocation and SCAN form the final combination. Quantitative results support LRU over FIFO and SCAN over FCFS for the representative workload. The design preserves fairness by not using SSTF as the default and prevents student work from being discarded by avoiding unsafe allocations before they occur.

7. Broader Considerations

Dimension	Design response
Reliability and safety	Exclusive writes, fair locking, permission checks, journaling, atomic rename, backups and checksums protect shared academic data.
Sustainability/economics	Demand paging, cache reuse, request merging and reduced seek movement improve hardware utilization and reduce unnecessary writes.
Accessibility/society	Bounded waiting, Round Robin for interactive CPU work and SCAN/C-SCAN fairness help prevent heavy users from blocking lighter users.
Ethics/professional duty	Apply least privilege, encrypt data in transit/at rest, minimize logs, define retention, audit administrative access and never monitor user data.
Transparency	Every default is visible; frame traces and seek sequences explain totals. No hidden AI service changes answers.
Reproducibility	Fixed inputs, deterministic tie rules, documented dependencies and automated assertions allow independent repetition.

Limitations

The simulator abstracts context-switch cost, TLB behavior, rotational latency, SSD characteristics and distributed-system failures. The proposed working-set thresholds are initial engineering values rather than measurements from a real CampusConnect deployment. Banker's Algorithm assumes declared maximum claims, which may not be available for arbitrary services.

8. Conclusion

The assignment requirements are satisfied through a working Streamlit trainer and a consolidated resource-management design. The synchronization scheme prevents races and writer starvation; the Banker's example is safe with sequence P1-P2-P3-P4; 8 GB memory contains 2,097,152 frames and a 64 MB process contains 16,384 pages; LRU gives 9 faults versus FIFO's 10; and SCAN gives 331 cylinders versus FCFS's 640. Indexed allocation with Unix-style inodes supports the mixed workload. Together, demand paging, working-set control, fair locking and SCAN provide a responsive and reproducible design.

9. Student Reflection

I learned that OS mechanisms cannot be selected independently. A process holding a lock may page fault, memory pressure changes response time, and file allocation affects the disk request stream seen by the scheduler. Quantitative traces made the trade-offs clearer than theory alone. I also learned that the algorithm with the smallest average is not automatically the best: fairness, starvation risk, recovery cost and transparency matter. With more time I would collect a real portal workload, add preemptive SJF and multi-level feedback queues, measure context-switch overhead, test concurrent POSIX-thread code under sanitizers, model SSD scheduling, and deploy the app with accessibility testing.

10. References

- [1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2015.
- [3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed., Pearson, 2018.
- [4] Python Software Foundation, *Python 3 Documentation*.
- [5] Streamlit Inc., *Streamlit Documentation*.
- [6] The Linux Kernel Documentation, *Memory Management and Block Layer*.
- [7] OpenAI Codex was used to assist with drafting and code generation; all numeric results were validated by executable tests.

Appendix A - Submitted Files

File	Purpose
app.py	Streamlit interface
algorithms.py	Pure algorithm implementations
test_algorithms.py	Automated validation
requirements.txt	Dependencies
README.md	Beginner-friendly run instructions

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Scheduling, Paging, Disk)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Streamlit, Python)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Process Scheduling	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – Page Replacement	CO3	PO1, PO2, PO3	L3/L4	6
Application – Disk Scheduling	CO4	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated OSLearn	CO1–CO4	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Streamlit / Python)	CO1–CO4	PO5	L3	10
Validation of results	CO1–CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

